Part A

A1: Response.status_code returns the HTTP status code from the server, 200 for a sucessful reuqest for 404 for server not found. len(response.content) returns the byte size of the response body, showing how much data the server sent back.

A2: response.text[:50] uses slicing to display only the first 50 characters of the response text. Changing 100 to 5 reduces the displayed output from 100 characters to 50, while the actual response remains unchanged

A3: This changes the query parameter ‘q’ in the URL and changes the requested URLs to ‘?q=banana’. The status code could stay the same if both requests are processed successfully. The response content or length may differ because the search results are different, different response length alone does not indicate a vulnerability

A4: The GET request sends the search term through URL query parameters, while the POST request sends it in the request body, so their status codes and response lengths may be different. A 405 response does not mean a vulnerability because it simply means the server does not allow the HTTP method for the requested endpoint.

Part B

B1: 4 times, 4 items, 4 iterations

B2: {'method': 'GET', 'endpoint': '/rest/products/search', 'status_code': 200, 'response_length': 1617, 'content_type': 'application/json; charset=utf-8'}

B3: [{'input': 'apple', 'status_code': 200, 'resposne_length': 9984}, {'input': 'banana', 'status_code': 200, 'resposne_length': 9984}]
A list of dictionaries organizes each test’s input, status code, and response length into structured records. This makes it easier to compare results, analyze data and generate reports or tables.

B4: Calling the function sneds a GET request using “banana” as a search terms and retrns the HTTP status code. ‘Search_word’ is a function parameter that receives the value passed into the function, such as ‘apple’ or ‘banana’. The function right now only returns the status code and returning a dictionary could provide extra information.

B5: A timeout prevents the scanner from waiting indefinitely if the server is slow or unresponsive. Handling ‘requests.RequestException’ allows the scanner to catch connection errors and other request-related failures gracefully instead of crashing.

Part C

Practice H: If the non-executable HTML string appears unchanged in the response, it means the input is reflected without output encoding, which is evidence of HTML injection. But does not prove XSS, because the browser may display the input as plain text rather than interpret it as HTML. An examination in the HTML context determines whether the input is rendered as executable markup to assess whether an XSS vulnerability exists.

Practice I: Both the normal input and probe input return HTTP 200, meaning the server processed both requests successfully. Comparing the probe response against a normal baseline is more informative because differences in response content, length or errors may reveal unusual behaviour that would not be obvious from examining the probe response alone.

Practice J:
1. “SQL syntax error”
2. “Unclosed quotation mark”

A matching phrase alone cannot prove successful SQL injection because the text might appear in a generic error message, documentation, or unrelated response content. Further investigation is needed to determine whether the input altered a database query and caused unintended behaviour.

Practice K: Use `response.headers.get("Server")` to display the server software information, if provided. It may reveal the web server name or version, which could help an attacker identify potential weaknesses. The presence of this header is not automatically a critical vulnerability because it does not prove the server is outdated or exploitable.

Practice L: The list contains only two alerts because the `seen` set prevents duplicate messages from being added. The second `"Missing CSP"` message is ignored because it has already been recorded.
In the final detector, two alerts should be considered the same finding when they share the same category, endpoint, and observation. This prevents duplicate alerts while allowing genuinely different findings to be reported separately.

Practice M: The CSV file has two column headings: `endpoint` and `status`.

Row 1: `/`, status `200`
Row 2: `/example`, status `404`

CSV is useful for the final lab report because it organizes scan results into a structured format that can be opened in spreadsheet applications, filtered, compared, and used to generate tables or summaries.

Part D

Task 3

1. Response Headers Present/Missing: On the local Juice Shop instance, security response headers such as Content-Security-Policy (CSP) or strict MIME-sniffing protections may be absent or loosely configured by default. Context matters because missing headers represent defense-in-depth gaps rather than direct remote code execution vectors. Their exploitability depends heavily on whether other mitigations are present.

2. Implementation Information Disclosure: The server may expose software stack footprints through headers. While helpful for fingerprinting versions during enumeration, disclosures are generally categorized as low-severity informational items unless tied to a known unpatched vulnerability in that specific build.

3. HTTPS/TLS Assessment Constraints: To assess an actual HTTPS deployment, we must audit certificate validity, cipher suite strengths, TLS protocol versions, and HSTS configuration with specialized tools or deep handshaking analysis.

4. Exploitability vs. Indicators: A missing header or a server error status code like 500 status code merely indicates abnormal application behavior or unhandled exceptions. It is not an automated confirmation of exploitability, we must do manual verification prove that an input can actually manipulate backend logic or execute arbitrary script code.

Task 4

```
costhm@costhm-IdeaPad-5-15IIL05:~$ docker ps
CONTAINER ID   IMAGE                  COMMAND                  CREATED          STATUS          PORTS
     NAMES
8ff06737ae46   bkimminich/juice-shop   "/nodejs/bin/node /j…"   17 minutes ago   Up 17 minutes   127.0.0.1:3000->3000/
tcp   fsct8561-juice
```

```
(Hard-Disk) costhm@costhm-IdeaPad-5-15IIL05:/media/costhm/Hard-Disk/FSCT8561-Labs-Alvin-Sin/Lab5$ python3 htt
p_scanner.py
[*] Starting HTTP Scanner on http://localhost:3000
[GET] / | Payload: None | Status: 200 | Length: 9984 bytes
[GET] /rest/products/search | Payload: {'q': 'apple'} | Status: 200 | Length: 1617 bytes
[GET] /rest/products/search | Payload: {'q': 'banana'} | Status: 200 | Length: 504 bytes
[POST] /rest/products/search | Payload: {'q': 'apple'} | Status: 500 | Length: 2442 bytes
[*] Scan complete. Results saved to http_scan_results.csv
(Hard-Disk) costhm@costhm-IdeaPad-5-15IIL05:/media/costhm/Hard-Disk/FSCT8561-Labs-Alvin-Sin/Lab5$ python3 vul
nerability_detector.py
[*] Starting Vulnerability Detector...

--- VULNERABILITY DETECTOR SUMMARY ---
[SQLi Indicator] Endpoint: /rest/products/search
  -> Observation: Probe ''')' resulted in status 500 and/or database error text.
  -> Status: Indicator found; manual verification required (not proof of exploit).

[Configuration Review] Endpoint: /
  -> Observation: Missing security header: Content-Security-Policy
  -> Status: Review relevance to application architecture.

[*] Findings exported to vulnerability_findings.csv
(Hard-Disk) costhm@costhm-IdeaPad-5-15IIL05:/media/costhm/Hard-Disk/FSCT8561-Labs-Alvin-Sin/Lab5$
```

Lab5 > http_scan_results.csv

```
1  endpoint,method,input_payload,status_code,response_length,content_type,server
2  /,GET,None,200,9984,text/html; charset=UTF-8,N/A
3  /rest/products/search,GET,{'q': 'apple'},200,1617,application/json; charset=utf-8,N/A
4  /rest/products/search,GET,{'q': 'banana'},200,504,application/json; charset=utf-8,N/A
5  /rest/products/search,POST,{'q': 'apple'},500,2442,text/html; charset=utf-8,N/A
```

Lab5 > vulnerability_findings.csv

```
1  category,endpoint,observation,review_status
2  SQLi Indicator,/rest/products/search,Probe ''')' resulted in status 500 and/or database error text.,Indicator found; manual verification required (not proof of exploit).
3  Configuration Review,/,Missing security header: Content-Security-Policy,Review relevance to application architecture.
```

```
(Hard-Disk) costhm@costhm-IdeaPad-5-15IIL05:/media/costhm/Hard-Disk/FSCT8561-Labs-Alvin-Sin/Lab5$ python3 --version
Python 3.12.3
```

No issue encountered

Task 5

Reflection and security analysis

The automated HTTP scanner observed routine functional responses, verifying that endpoints like / and /rest/products/search return standard status codes (200) alongside predictable content lengths and MIME types. Meanwhile, the vulnerability detector flagged specific indicators, the unconditional reflection of text markers and input probes, like such as single quotes ' resulting in application exceptions or altered behavior. Crucially, these outputs represent potential indicators rather than confirmed vulnerabilities; reflection alone does not equate to active cross-site scripting (XSS) if the client safely encodes output, and server errors do not automatically prove successful SQL injection without verifying backend data extraction.

If tasked as the security lead for Juice Shop, the immediate priority would be investigating how user-supplied search parameters are handled and sanitized before interacting with backend queries or rendering back to the Document Object Model (DOM). Securing input validation layers and implementing context-aware output encoding would mitigate both injection and reflection vectors.

This lab builds directly upon Lab 4 by shifting focus from passive packet capture where we observed raw network streams on the wire, to active application-layer testing using Python. While a packet capture on an unencrypted HTTP deployment reveals plaintext parameters and response bodies seamlessly, an HTTPS deployment would encrypt payload contents via TLS, hiding query strings and POST bodies from passive network sniffers while still allowing authorized HTTP clients to inspect responses programmatically.

In summary, automated scanning tools are triage mechanisms designed to reduce manual workload. They generate flags based on static heuristics and syntactic matching. Because automated tools lack contextual awareness of application defenses, reporting unverified indicators as confirmed vulnerabilities risks false positives, necessitating rigorous manual confirmation before remediation or escalation.